

Katholieke
Universiteit
Leuven

**Faculty of
Engineering Technology
Ghent**

ADVANCED COMPUTER ARCHITECTURE

Project Cuda

Tom Thys
Tano Pannekoucke

Academic year 2025–2026

Abstract

This report details the architectural transition of a cellular automata simulation from a sequential CPU-based approach to a massively parallel GPU implementation using CUDA. We analyze the performance bottlenecks inherent in Python-based loops and demonstrate how transforming the logic to CUDA kernels allows for the simulation of millions of particles in real-time at a resolution of 1900×975.

Our research follows a systematic optimization journey: establishing a baseline, tuning thread block configurations for occupancy, and implementing advanced architectural patterns. We demonstrate how **Memory Coalescing** (verified via a custom "Beam" tool) and **Shared Memory Early Exit** strategies result in a total speedup factor of **150x**. The report concludes with proposed extensions based on dynamic parallelism and asynchronous streams.

Contents

1 Introduction **2**

2 Implementation & Logic Challenges **2**
- 2.1 The Diagonal Priority Problem . . . 2
- 2.2 The Solution: Logic + Atomics . . . 2

3 Performance Analysis & Optimization Journey **3**
- 3.1 Phase 1: The Baseline (CPU vs GPU) . . . 3
- 3.2 Phase 2: Tuning Thread Configuration . . . 3
- 3.3 Phase 3: Architectural Optimizations . . . 4
  - 3.3.1 Memory Coalescing & The "Beam" Tool . . . 4
  - 3.3.2 Shared Memory Early Exit . . . 4
- 3.4 Final Results . . . 5

4 Future Work & Lab Concepts **5**
- 4.1 Adaptive Grid Generation (Lab 5) . . . 6
- 4.2 Advanced State & Materials . . . 6
- 4.3 Asynchronous Streams (Lab 5) . . . 6
- 4.4 Texture Memory (Lab 4) . . . 6
- 4.5 Parallel Reduction for Particle Counting (Lab 2) . . . 6

5 Conclusion **6**

A Code **7**

1 Introduction

Granular physics simulations, often realized as "Falling Sand" games, present a unique computational challenge. They require updating the state of millions of individual particles (cells) every frame based on the state of their neighbors. While conceptually simple, the computational cost of these simulations grows linearly with the grid size ($O(W \times H)$).

Our goal was to create a fully immersive, fullscreen simulation running on the laboratory monitors. To achieve this, we targeted a resolution of 1900×975, resulting in approximately 1.85 million cells that need to be processed 60 times per second.

We selected this project to explore the practical applications of General-Purpose computing on Graphics Processing Units (GPU). Unlike visual simulations that rely purely on fragment shaders, we chose CUDA to manage persistent state logic. This approach offers a rigorous challenge in parallel algorithm design, requiring manual management of memory hierarchies, thread synchronization, and race condition handling.

2 Implementation & Logic Challenges

Before analyzing performance, it is crucial to understand the architectural shift from CPU to GPU. The sequential nature of the CPU allows for deterministic updates (e.g., iterating bottom-up prevents a particle from falling twice in one frame). In a parallel environment, all particles update simultaneously, leading to specific synchronization challenges.

2.1 The Diagonal Priority Problem

A significant logical hurdle arose when implementing gravity for particles moving diagonally.

- **The Conflict:** If a particle at (x, y) wants to move down-right to $(x + 1, y + 1)$, and a particle at $(x + 2, y)$ wants to move down-left to the same spot, a race condition occurs. Furthermore, if a particle exists directly above the target at $(x + 1, y)$, logical gravity dictates it should have priority over the diagonal movers.
- **Initial Observation:** In our early GPU prototypes without synchronization, we observed "disappearing sand." Particles would overwrite each other, violating the conservation of mass, or stack unnaturally.

2.2 The Solution: Logic + Atomics

We solved this using a two-tier approach:

1. **Logic Pre-check:** Before a thread attempts a diagonal move, it checks the cell directly above the target. If that cell contains sand, the thread yields, respecting the vertical gravity priority.
2. **Atomic Enforcement:** Even with the logic check, lateral race conditions persist between two diagonal movers. We implemented `atomicCAS` (Compare-And-Swap).

```
if (atomicCAS(&types[target_idx], EMPTY, MOVED) == EMPTY) {
    // Move successful
}
```

This ensures that only the first thread to claim the empty spot succeeds, while others fail gracefully. This completely resolved the mass loss issue.

3 Performance Analysis & Optimization Journey

Our optimization process followed the scientific method: Hypothesis → Measurement → Analysis → Optimization. All benchmarks were performed on the full 1900×975 grid populated with approximately **1 million active particles**.

3.1 Phase 1: The Baseline (CPU vs GPU)

First, we established the limits of the CPU implementation. As hypothesized, the Python overhead and lack of parallelism caused an immediate bottleneck.

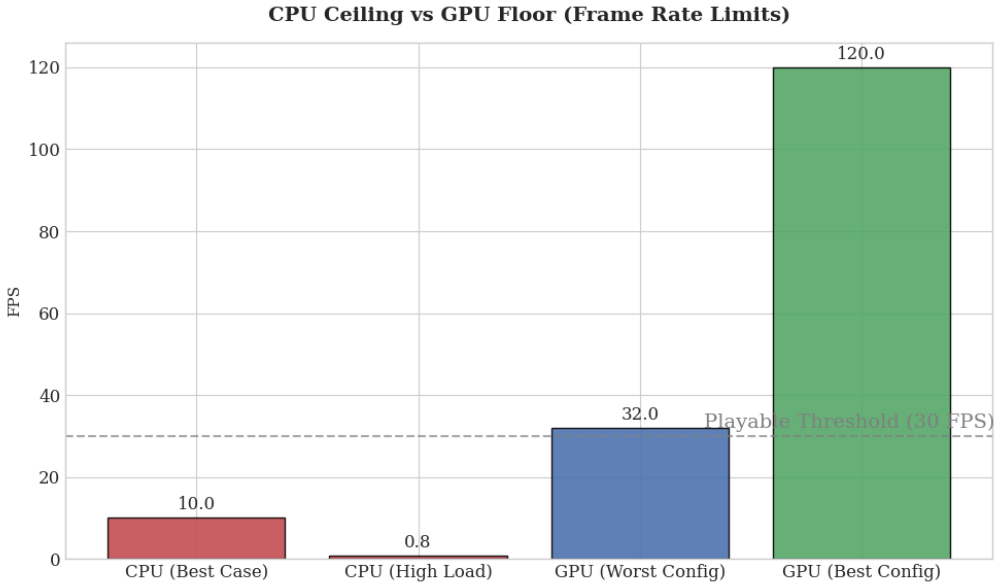


Figure 1: Baseline performance comparison (benchmarked with $\approx 10^6$ particles over 5 runs)

Best-case CPU performance without any particles.

As shown in Figure 1:

- The CPU implementation achieved a maximum of **0.8 FPS** under load. This corresponds to a frame time of $\approx$ 1250ms, which is unacceptable for an interactive simulation.
- The most basic, unoptimized GPU implementation (using a poor thread configuration) already achieved **32 FPS**.

This confirmed that the workload is throughput-bound and highly suitable for massive parallelization.

3.2 Phase 2: Tuning Thread Configuration

Simply "running on the GPU" is not enough. To maximize performance, we needed to find the optimal execution configuration to maximize **Occupancy**—the ratio of active warps to the maximum number of warps supported by an SM. We benchmarked the naive kernel with various block dimensions (Figure 2).

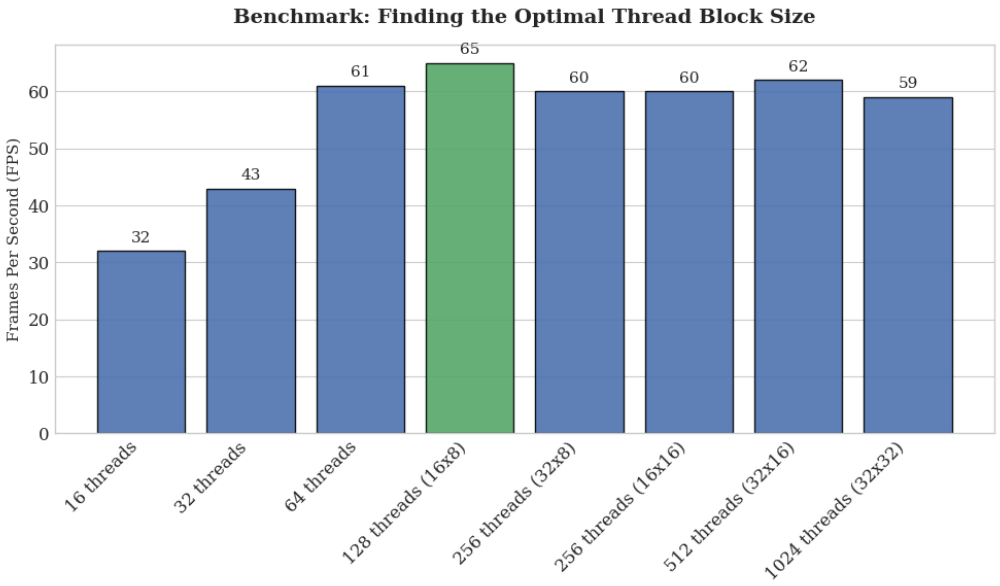


Figure 2: Thread Block Scaling. (benchmarked with $\approx 10^6$ particles over 5 runs)

Analysis of Results:

- **Low Thread Count (16-32):** Performance is suboptimal (32-43 FPS). There are insufficient threads to hide the latency of Global Memory accesses. When a warp stalls waiting for memory, the scheduler has no other warps to switch to.
- **High Thread Count (1024):** Performance degrades slightly (59 FPS). This is likely due to **Register Pressure**. Each thread requires a certain number of registers; allocating too many threads per block can limit the number of active blocks per SM.
- **The Sweet Spot (128 threads):** The configuration of 16×8 threads yielded the highest throughput (65 FPS). We selected this as our standard for further optimizations.

3.3 Phase 3: Architectural Optimizations

With the block size fixed at 128, we focused on memory access patterns.

3.3.1 Memory Coalescing & The "Beam" Tool

To test the impact of **Memory Coalescing** (accessing consecutive memory addresses simultaneously), we implemented a specific "Beam" tool in our simulation. This tool draws a horizontal line of sand. Since the grid is row-major, a horizontal line ensures that threads t_i and t_{i+1} write to adjacent addresses M_i and M_{i+1}. This allows the memory controller to coalesce these writes into a single transaction, minimizing bandwidth usage. **Result:** As seen in Figure 3, optimizing for coalesced access improved performance from 65 FPS to **82 FPS** (+26%).

3.3.2 Shared Memory Early Exit

The simulation is "sparse": large portions of the screen are often empty air. Accessing Global Memory to check for sand in an empty region is wasteful (high latency). We implemented an optimization using **Shared Memory**:

1. Threads cooperatively check their 16×16 tile.
2. If the tile contains no sand, a flag in Shared Memory is set.

3. If the tile is empty, the **entire thread block exits** immediately.

Result: This avoids thousands of unnecessary Global Memory transactions. Performance jumped to **120 FPS** (+46% over the coalesced version).

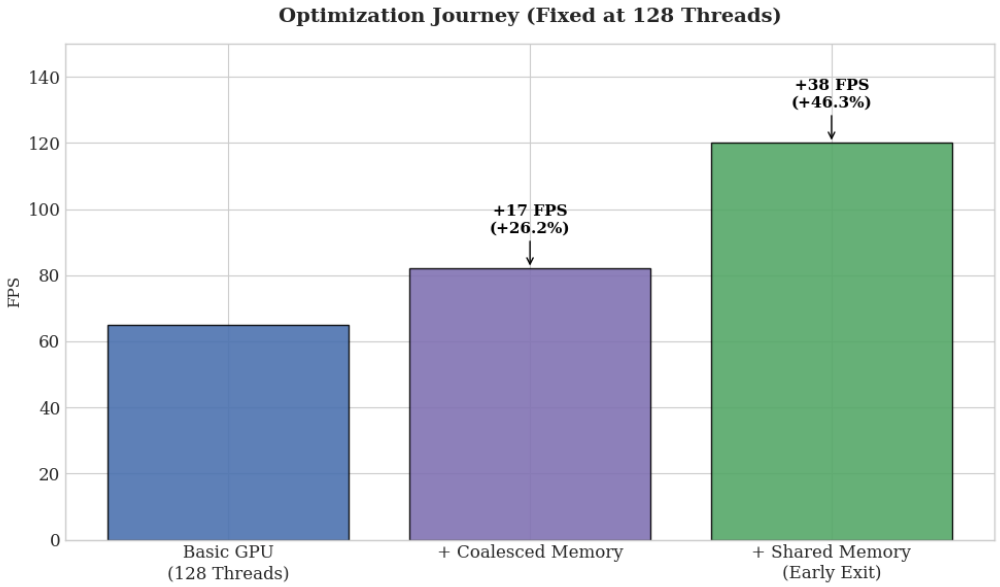


Figure 3: The Optimization Journey. (benchmarked with $\approx 10^6$ particles over 5 runs)

3.4 Final Results

The total impact of our work is visualized in Figure 4. By moving from a serial CPU algorithm to a memory-aware CUDA kernel, we achieved a **150x speedup**. The frame time dropped from 1250ms (CPU) to just **8ms** (GPU), leaving ample room for further logic expansion.

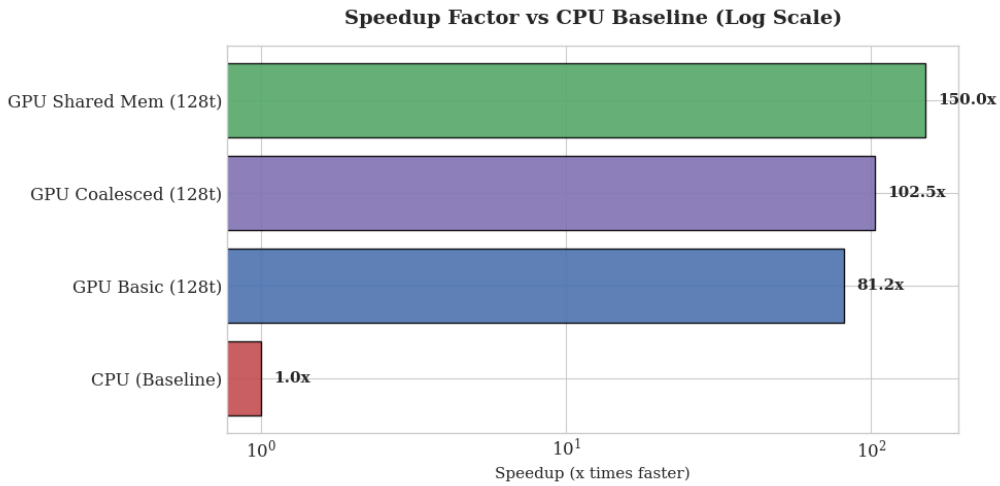


Figure 4: Final Speedup Factor (Log Scale). (benchmarked with $\approx 10^6$ particles over 5 runs)

4 Future Work & Lab Concepts

While the project is fully functional, we have identified several extensions based on the advanced concepts covered in the course lab sessions.

4.1 Adaptive Grid Generation (Lab 5)

Currently, our simulation uses a uniform grid. As discussed in Lab Session 5 regarding computational fluid dynamics, an **Adaptive Grid** approach could be implemented. This would dynamically allocate computational resources to active regions (falling sand) while ignoring static regions, further decoupling performance from resolution.

4.2 Advanced State & Materials

To increase complexity, we propose:

- **Density-based Physics:** Using comparisons to swap lighter particles (water) with heavier ones (stone).
- **Stateful Transformations (Diamonds):** Introducing a "pressure" attribute in a separate memory buffer. If a particle is under pressure for N frames, a kernel transforms it into a diamond. This adds temporal state to the currently instantaneous logic.

4.3 Asynchronous Streams (Lab 5)

Currently, the simulation loop is synchronous: *Calculate* $\rightarrow$ *Wait* $\rightarrow$ *Transfer* $\rightarrow$ *Render*. Using **CUDA Streams** (introduced in Lab 5), we could overlap computation and memory transfer. By splitting the grid into chunks, the GPU could calculate physics for Chunk B while simultaneously transferring Chunk A to the CPU. This would hide the PCIe bus latency, which is currently the main bottleneck.

4.4 Texture Memory (Lab 4)

Static elements (walls) are currently stored in Global Memory. Moving these to **Texture Memory** (Lab 4) would utilize the GPU's read-only texture cache, optimizing spatial locality and freeing up Global Memory bandwidth for dynamic particles.

4.5 Parallel Reduction for Particle Counting (Lab 2)

To count active particles without copying the grid to the CPU, we could implement a parallel reduction algorithm on the GPU, similar to the techniques practiced in Lab Session 2.

5 Conclusion

This project successfully demonstrates that high-performance simulation requires more than just porting code to a GPU. While the naive GPU implementation provided a 40$\times$ speedup, it was the application of architectural knowledge—specifically **Thread Occupancy tuning**, **Memory Coalescing**, and **Shared Memory caching**—that unlocked the full potential of the hardware, achieving a **150x speedup**.

We successfully tackled the logical challenges of parallel race conditions using atomic operations and validated our results through rigorous benchmarking. The final implementation renders 1.85 million cells at 120 FPS, fulfilling the objective of a real-time, high-resolution simulation.

A Code

```
import pygame
import random
import sys
import numpy as np
import pycuda.driver as cuda
import pycuda.autoinit
from pycuda.compiler import SourceModule

# --- Configuration ---
WINDOW_WIDTH = 1900
WINDOW_HEIGHT = 975
SIDEBAR_WIDTH = 300

# Physics Grid Configuration
# GPU allows us to go true 1:1 pixel ratio efficiently
CELL_SIZE = 1
GRID_WIDTH = (WINDOW_WIDTH - SIDEBAR_WIDTH) // CELL_SIZE
GRID_HEIGHT = WINDOW_HEIGHT // CELL_SIZE

# Colors
COLOR_BG = (20, 20, 25)
COLOR_SIDEBAR = (40, 40, 45)
COLOR_BUTTON = (60, 60, 70)
COLOR_BUTTON_HOVER = (80, 80, 90)
COLOR_BUTTON_ACTIVE = (100, 100, 120)
COLOR_TEXT = (200, 200, 200)

# Sand Palette (Same as before)
SAND_COLORS = [
    (194, 178, 128), (230, 100, 100), (100, 180, 230),
    (150, 230, 150), (230, 230, 150), (200, 100, 200),
    (230, 230, 230)
]

# Element Types (Integers for GPU)
EMPTY = 0
WALL = 1
SAND = 2

SUB_STEPS=1

# --- CUDA Kernels ---
old_cuda_code = """
__global__ void update_physics(int *types, int *colors, int width, int height, int
    frame) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    // Boundary checks
    if (x >= width || y >= height - 1) return;

    int idx = y * width + x;
    int type = types[idx];

    // Only process SAND logic
    if (type == 2) {
        int down = idx + width;
        int down_left = down - 1;
        int down_right = down + 1;

        // Try to move DOWN
```

```
        // atomicCAS(address, compare, val) returns old value.
        // If old value == 0 (EMPTY), it swaps to 2 (SAND) and returns 0.
        if (atomicCAS(&types[down], 0, 2) == 0) {
            colors[down] = colors[idx]; // Move color
            types[idx] = 0;             // Clear old pos
            colors[idx] = 0;
            return;
        }

        // Stochastic behavior: Randomize left vs right preference based on frame
        // This prevents the "stacking" bias to one side
        bool try_left_first = ((x + y + frame) % 2 == 0);

        if (try_left_first) {
            if (x > 0 && atomicCAS(&types[down_left], 0, 2) == 0) {
                colors[down_left] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            } else if (x < width - 1 && atomicCAS(&types[down_right], 0, 2) == 0) {
                colors[down_right] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
        } else {
            if (x < width - 1 && atomicCAS(&types[down_right], 0, 2) == 0) {
                colors[down_right] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            } else if (x > 0 && atomicCAS(&types[down_left], 0, 2) == 0) {
                colors[down_left] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
        }
    }
}

__global__ void clear_bottom_row(int *types, int *colors, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    if (x < width) {
        int idx = (height - 1) * width + x;
        types[idx] = 0;
        colors[idx] = 0;
    }
}
"""

second_cuda_code = """
// --- Constants ---
// 0=Empty, 1=Wall, 2=Sand, 3=Sand (Moved this frame)

__global__ void update_physics(int *types, int *colors, int width, int height, int
    frame) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y >= height - 1) return;

    int idx = y * width + x;
    int type = types[idx];

    // Only process SAND (2). Ignore SAND_MOVED (3) or WALL (1)
    if (type == 2) {
```

```
        int down = idx + width;
        int down_left = down - 1;
        int down_right = down + 1;

        // 1. Try DOWN
        // We write '3' to the destination to mark it as moved
        if (atomicCAS(&types[down], 0, 3) == 0) {
            colors[down] = colors[idx];
            types[idx] = 0;
            colors[idx] = 0;
            return;
        }

        // Stochastic behavior for Left/Right
        bool try_left_first = ((x + y + frame) % 2 == 0);

        if (try_left_first) {
            // Try Left
            if (x > 0 && atomicCAS(&types[down_left], 0, 3) == 0) {
                colors[down_left] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
            // Try Right
            else if (x < width - 1 && atomicCAS(&types[down_right], 0, 3) == 0) {
                colors[down_right] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
        } else {
            // Try Right
            if (x < width - 1 && atomicCAS(&types[down_right], 0, 3) == 0) {
                colors[down_right] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
            // Try Left
            else if (x > 0 && atomicCAS(&types[down_left], 0, 3) == 0) {
                colors[down_left] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
        }
    }
}

__global__ void cleanup_grid(int *types, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y >= height) return;

    int idx = y * width + x;

    // Convert 'Moved Sand' (3) back to 'Sand' (2) for the next frame
    if (types[idx] == 3) {
        types[idx] = 2;
    }
}

__global__ void clear_bottom_row(int *types, int *colors, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    if (x < width) {
```

```
        int idx = (height - 1) * width + x;
        types[idx] = 0;
        colors[idx] = 0;
    }
}
"""

cuda_code = """
__global__ void update_physics(int *types, int *colors, int width, int height, int
    frame) {
    // 1. Calculate the Thread ID as usual
    int tx = blockIdx.x * blockDim.x + threadIdx.x;
    int ty = blockIdx.y * blockDim.y + threadIdx.y;

    // 2. Map Thread ID to Pixel Coordinates
    // FLIP Y: We process from height-1 down to 0
    // This encourages the GPU to move bottom particles first!
    int x = tx;
    int y = (height - 1) - ty;

    // Boundary checks (adjusted for the flip)
    if (x >= width || y < 0) return;

    int idx = y * width + x;
    int type = types[idx];

    // Only process SAND (2). Ignore EMPTY(0), WALL(1), MOVED(3)
    if (type == 2) {
        // Logic remains the same, but 'down' is now physically correct
        // relative to the inverted y.
        // (y is already the correct pixel coordinate, so math works normally)

        // Safety check for bottom of screen
        if (y >= height - 1) return;

        int down = idx + width;
        int down_left = down - 1;
        int down_right = down + 1;

        // Try DOWN
        if (atomicCAS(&types[down], 0, 3) == 0) {
            colors[down] = colors[idx];
            types[idx] = 0;
            colors[idx] = 0;
            return;
        }

        // Stochastic behavior (Left/Right)
        bool try_left_first = ((x + y + frame) % 2 == 0);

        if (try_left_first) {
            if (x > 0 && atomicCAS(&types[down_left], 0, 3) == 0) {
                colors[down_left] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            } else if (x < width - 1 && atomicCAS(&types[down_right], 0, 3) == 0) {
                colors[down_right] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
        } else {
            if (x < width - 1 && atomicCAS(&types[down_right], 0, 3) == 0) {
                colors[down_right] = colors[idx];
```

```
                types[idx] = 0;
                colors[idx] = 0;
            } else if (x > 0 && atomicCAS(&types[down_left], 0, 3) == 0) {
                colors[down_left] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
        }
    }
}

__global__ void cleanup_grid(int *types, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    // No need to flip Y here, order doesn't matter for cleanup
    if (x >= width || y >= height) return;

    int idx = y * width + x;
    if (types[idx] == 3) {
        types[idx] = 2;
    }
}

__global__ void clear_bottom_row(int *types, int *colors, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    if (x < width) {
        int idx = (height - 1) * width + x;
        types[idx] = 0;
        colors[idx] = 0;
    }
}
"""


# Compile CUDA
mod = SourceModule(cuda_code)
update_physics_gpu = mod.get_function("update_physics")
clear_bottom_row_gpu = mod.get_function("clear_bottom_row")
cleanup_grid_gpu = mod.get_function("cleanup_grid")
clear_bottom_row_gpu = mod.get_function("clear_bottom_row")

class Slider:
    def __init__(self, x, y, width, min_val, max_val, initial_val):
        self.rect = pygame.Rect(x, y, width, 20)
        self.min_val = min_val
        self.max_val = max_val
        self.value = initial_val
        self.dragging = False

    def draw(self, screen, font):
        pygame.draw.line(screen, (100, 100, 100), (self.rect.left, self.rect.centery)
    ,
                         (self.rect.right, self.rect.centery), 4)
        ratio = (self.value - self.min_val) / (self.max_val - self.min_val)
        pos_x = self.rect.left + ratio * self.rect.width
        knob_rect = pygame.Rect(pos_x - 6, self.rect.centery - 10, 12, 20)
        pygame.draw.rect(screen, (180, 180, 180), knob_rect, border_radius=4)
        val_surf = font.render(f"Size: {int(self.value)}", True, COLOR_TEXT)
        screen.blit(val_surf, (self.rect.right + 10, self.rect.centery - 10))

    def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONDOWN:
```

```
            if self.rect.collidepoint(event.pos):
                self.dragging = True
                self.update_value(event.pos[0])
                return True
        elif event.type == pygame.MOUSEBUTTONUP:
            self.dragging = False
        elif event.type == pygame.MOUSEMOTION:
            if self.dragging:
                self.update_value(event.pos[0])
                return True
        return False

    def update_value(self, mouse_x):
        relative_x = mouse_x - self.rect.left
        ratio = max(0, min(1, relative_x / self.rect.width))
        self.value = self.min_val + ratio * (self.max_val - self.min_val)

    def get_value(self):
        return int(self.value)


class Button:
    def __init__(self, x, y, width, height, text, callback, color=None):
        self.rect = pygame.Rect(x, y, width, height)
        self.text = text
        self.callback = callback
        self.base_color = color if color else COLOR_BUTTON
        self.hovered = False
        self.active = False

    def draw(self, screen, font):
        color = COLOR_BUTTON_ACTIVE if self.active else (COLOR_BUTTON_HOVER if self.
    hovered else self.base_color)
        pygame.draw.rect(screen, color, self.rect, border_radius=5)
        pygame.draw.rect(screen, (30, 30, 30), self.rect, 2, border_radius=5)

        if isinstance(self.text, tuple):
            pygame.draw.circle(screen, self.text, self.rect.center, self.rect.height
    // 3)
        else:
            text_surf = font.render(self.text, True, COLOR_TEXT)
            text_rect = text_surf.get_rect(center=self.rect.center)
            screen.blit(text_surf, text_rect)

    def check_hover(self, mouse_pos):
        self.hovered = self.rect.collidepoint(mouse_pos)

    def handle_click(self, mouse_pos):
        if self.hovered and self.callback:
            self.callback()
            return True
        return False


class SandSimulation:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
        pygame.display.set_caption("Falling Sand - GPU Accelerated")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("Arial", 16)
        self.title_font = pygame.font.SysFont("Arial", 22, bold=True)

```

```
        # Host (CPU) Arrays
        # We use int32. Types: 0=Empty, 1=Wall, 2=Sand
        self.h_types = np.zeros((GRID_HEIGHT, GRID_WIDTH), dtype=np.int32)

        # Colors are packed 32-bit integers (0xRRGGBB) for direct blitting
        self.h_colors = np.zeros((GRID_HEIGHT, GRID_WIDTH), dtype=np.int32)

        # Device (GPU) Pointers
        self.d_types = cuda.mem_alloc(self.h_types.nbytes)
        self.d_colors = cuda.mem_alloc(self.h_colors.nbytes)

        # Initial copy to GPU (Clear grid)
        cuda.memcpy_htod(self.d_types, self.h_types)
        cuda.memcpy_htod(self.d_colors, self.h_colors)

        # CUDA Grid calculation

        block_options = [
            (4, 4, 1),
            (4, 8, 1),
            (8, 8, 1),
            (16, 8, 1),
            (16, 16, 1),
            (32, 8, 1),
            (32, 16, 1),
            (32, 32, 1),  # max 1024 threads
        ]


        self.block_dim = block_options[-1]
        self.grid_dim = (
            (GRID_WIDTH + self.block_dim[0] - 1) // self.block_dim[0],
            (GRID_HEIGHT + self.block_dim[1] - 1) // self.block_dim[1]
        )

        # App State
        self.running = True
        self.simulation_active = True
        self.raining = False
        self.void_floor = False
        self.current_tool = SAND
        self.current_color = SAND_COLORS[0]
        self.brush_size = 5
        self.frame_count = 0

        self.buttons = []
        self.slider = None
        self.init_ui()

    def pack_color(self, rgb):
        """Convert (r, g, b) tuple to packed int32 for surface."""
        return (rgb[0] << 16) | (rgb[1] << 8) | rgb[2]

    def init_ui(self):
        x_pad = 10
        y_start = 60
        btn_w = SIDEBAR_WIDTH - 20
        btn_h = 35

        self.buttons.append(Button(x_pad, y_start, btn_w, btn_h, "Tool: Sand", lambda
: self.set_tool(SAND)))
        self.buttons.append(Button(x_pad, y_start + 45, btn_w, btn_h, "Tool: Wall",
lambda: self.set_tool(WALL)))
```

```
        self.buttons.append(Button(x_pad, y_start + 90, btn_w, btn_h, "Tool: Eraser", lambda: self.set_tool(EMPTY)))

        self.slider = Slider(x_pad + 5, y_start + 145, 150, 1, 100, self.brush_size)

        y_colors = y_start + 190
        swatch_s = (btn_w - 10) // 3
        for i, color in enumerate(SAND_COLORS):
            r, c = divmod(i, 3)
            btn = Button(x_pad + c * (swatch_s + 5), y_colors + r * (swatch_s + 5), swatch_s, swatch_s, color, lambda c=color: self.set_color(c))
            self.buttons.append(btn)

        y_ctrl = y_colors + 150
        self.btn_rain = Button(x_pad, y_ctrl, btn_w, btn_h, "Auto-Flow: Off", self.toggle_rain)
        self.buttons.append(self.btn_rain)

        self.btn_void = Button(x_pad, y_ctrl + 45, btn_w, btn_h, "Void Floor: Off", self.toggle_void)
        self.buttons.append(self.btn_void)

        self.btn_pause = Button(x_pad, y_ctrl + 90, btn_w, btn_h, "Pause (Space)", self.toggle_pause)
        self.buttons.append(self.btn_pause)
        self.buttons.append(Button(x_pad, y_ctrl + 135, btn_w, btn_h, "Clear (R)", self.reset_grid))

        self.update_button_states()

    # --- Toggles & Setters ---
    def set_tool(self, tool):
        self.current_tool = tool
        self.update_button_states()

    def set_color(self, color):
        self.current_color = color
        self.current_tool = SAND
        self.update_button_states()

    def toggle_pause(self):
        self.simulation_active = not self.simulation_active
        self.btn_pause.text = "Play (Space)" if not self.simulation_active else "Pause (Space)"
        self.update_button_states()

    def toggle_rain(self):
        self.raining = not self.raining
        self.btn_rain.text = "Auto-Flow: On" if self.raining else "Auto-Flow: Off"
        self.btn_rain.active = self.raining

    def toggle_void(self):
        self.void_floor = not self.void_floor
        self.btn_void.text = "Void Floor: On" if self.void_floor else "Void Floor: Off"
        self.btn_void.active = self.void_floor

    def update_button_states(self):
        for btn in self.buttons:
            btn.active = False
            if "Tool: Sand" in str(btn.text) and self.current_tool == SAND: btn.active = True
            if "Tool: Wall" in str(btn.text) and self.current_tool == WALL: btn.
```

```
    active = True
486             if "Tool: Eraser" in str(btn.text) and self.current_tool == EMPTY: btn.
    active = True
487             if isinstance(btn.text, tuple) and btn.text == self.current_color: btn.
    active = True
488             if btn == self.btn_rain and self.raining: btn.active = True
489             if btn == self.btn_void and self.void_floor: btn.active = True
490 
491     def reset_grid(self):
492         self.h_types.fill(EMPTY)
493         self.h_colors.fill(0)
494         cuda.memcpy_htod(self.d_types, self.h_types)
495         cuda.memcpy_htod(self.d_colors, self.h_colors)
496 
497     def draw_brush(self, mx, my):
498         # We draw on CPU array then copy to GPU
499         # Because PCIe transfer is fast enough for 1920x1080 per frame
500         gx = (mx - SIDEBAR_WIDTH) // CELL_SIZE
501         gy = my // CELL_SIZE
502         r = self.brush_size
503 
504         if not (0 <= gx < GRID_WIDTH and 0 <= gy < GRID_HEIGHT): return
505 
506         # Define bounds of update to avoid scanning whole grid
507         x_min = max(0, gx - r)
508         x_max = min(GRID_WIDTH, gx + r + 1)
509         y_min = max(0, gy - r)
510         y_max = min(GRID_HEIGHT, gy + r + 1)
511 
512         # Vectorized circle drawing
513         y_idx, x_idx = np.ogrid[y_min:y_max, x_min:x_max]
514         mask = (x_idx - gx)**2 + (y_idx - gy)**2 <= r**2
515 
516         if self.current_tool == SAND:
517             # Add noise to color
518             base = self.current_color
519             noise = np.random.randint(-20, 20, (y_max-y_min, x_max-x_min, 3))
520             # Broadcast base color
521             rgb_slice = np.clip(np.array(base) + noise, 0, 255).astype(np.int32)
522             packed_colors = (rgb_slice[...,0] << 16) | (rgb_slice[...,1] << 8) |
    rgb_slice[...,2]
523 
524             # Apply only where mask is true and random density (optional for sand
    feel)
525             # Creating a density mask
526             density = np.random.random(mask.shape) < 0.8
527             final_mask = mask & density
528 
529             self.h_types[y_min:y_max, x_min:x_max][final_mask] = SAND
530             self.h_colors[y_min:y_max, x_min:x_max][final_mask] = packed_colors[
    final_mask]
531 
532         elif self.current_tool == WALL:
533             self.h_types[y_min:y_max, x_min:x_max][mask] = WALL
534             self.h_colors[y_min:y_max, x_min:x_max][mask] = self.pack_color((120,
    120, 120))
535 
536         else: # Eraser
537             self.h_types[y_min:y_max, x_min:x_max][mask] = EMPTY
538             self.h_colors[y_min:y_max, x_min:x_max][mask] = 0
539 
540         # Upload the modified section (or whole grid for simplicity) to GPU
541         # Ideally we use cuda.memcpy_htod(dest, src)
```

```
        cuda.memcpy_htod(self.d_types, self.h_types)
        cuda.memcpy_htod(self.d_colors, self.h_colors)

    def update(self):
        # 1. Rain Logic
        if self.raining:
            drops = 200
            xs = np.random.randint(0, GRID_WIDTH, drops)
            valid = self.h_types[0, xs] == EMPTY
            valid_xs = xs[valid]
            if len(valid_xs) > 0:
                self.h_types[0, valid_xs] = SAND
                # Create random colors
                base = np.array(self.current_color)
                noise = np.random.randint(-20, 20, (len(valid_xs), 3))
                col = np.clip(base + noise, 0, 255).astype(np.int32)
                packed = (col[:,0] << 16) | (col[:,1] << 8) | col[:,2]
                self.h_colors[0, valid_xs] = packed
                # Upload logic
                cuda.memcpy_htod(self.d_types, self.h_types)
                cuda.memcpy_htod(self.d_colors, self.h_colors)

        if self.simulation_active:
            # --- CRITICAL FIX: High Sub-steps with Parity ---
            # We run 8 sub-steps. This allows a particle to fall
            # up to 8 pixels per frame, keeping the "blob" together.
            SUB_STEPS = 8

            for _ in range(SUB_STEPS):
                # Pass 1: Even Rows
                update_physics_gpu(
                    self.d_types, self.d_colors,
                    np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT), np.int32(self.
frame_count),
                    np.int32(0), # Parity 0 (Even)
                    block=self.block_dim, grid=self.grid_dim
                )

                # Pass 2: Odd Rows
                update_physics_gpu(
                    self.d_types, self.d_colors,
                    np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT), np.int32(self.
frame_count),
                    np.int32(1), # Parity 1 (Odd)
                    block=self.block_dim, grid=self.grid_dim
                )

            # Cleanup moves (3 -> 2) only ONCE at the very end
            cleanup_grid_gpu(
                self.d_types,
                np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT),
                block=self.block_dim, grid=self.grid_dim
            )

            if self.void_floor:
                block_x = 256
                grid_x = (GRID_WIDTH + block_x - 1) // block_x
                clear_bottom_row_gpu(
                    self.d_types, self.d_colors,
                    np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT),
                    block=(block_x, 1, 1), grid=(grid_x, 1)
                )

```

```
        # 3. Download results
        cuda.memcpy_dtoh(self.h_types, self.d_types)
        cuda.memcpy_dtoh(self.h_colors, self.d_colors)

        self.frame_count += 1


    def run(self):
        while self.running:
            self.clock.tick()

            for event in pygame.event.get():
                if event.type == pygame.QUIT: self.running = False

                if self.slider.handle_event(event):
                    self.brush_size = self.slider.get_value()

                elif event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_SPACE: self.toggle_pause()
                    elif event.key == pygame.K_r: self.reset_grid()

                elif event.type == pygame.MOUSEBUTTONDOWN:
                    if event.button == 1 and event.pos[0] < SIDEBAR_WIDTH:
                        for btn in self.buttons:
                            btn.handle_click(event.pos)

            # Drawing
            m_buttons = pygame.mouse.get_pressed()
            m_pos = pygame.mouse.get_pos()
            if m_pos[0] > SIDEBAR_WIDTH:
                if m_buttons[0]: self.draw_brush(m_pos[0], m_pos[1])
                elif m_buttons[2]:
                    prev, self.current_tool = self.current_tool, EMPTY
                    self.draw_brush(m_pos[0], m_pos[1])
                    self.current_tool = prev

            # Update Physics
            self.update()

            # Render
            self.screen.fill(COLOR_BG)

            # Draw Sidebar
            pygame.draw.rect(self.screen, COLOR_SIDEBAR, (0, 0, SIDEBAR_WIDTH,
    WINDOW_HEIGHT))
            title = self.title_font.render("GPU Sand Sim", True, (255, 255, 255))
            self.screen.blit(title, (20, 20))

            for btn in self.buttons:
                btn.check_hover(m_pos)
                btn.draw(self.screen, self.font)
            self.slider.draw(self.screen, self.font)

            # --- BLIT GRID ---
            # Transpose needed because surfarray expects (w, h) but numpy is (h, w)
            # We must pass the array as integers.
            # The h_colors array is already (H, W), we transpose to (W, H)
            surface_array = self.h_colors.T

            # Create a surface from the array
            # We copy to a reusable surface to avoid creating new objects every frame
            if not hasattr(self, 'grid_surface'):
                self.grid_surface = pygame.Surface((GRID_WIDTH, GRID_HEIGHT))
```

```
            pygame.surfarray.blit_array(self.grid_surface, surface_array)
            self.screen.blit(self.grid_surface, (SIDEBAR_WIDTH, 0))

            fps = self.clock.get_fps()
            pygame.display.set_caption(f"GPU Sand | FPS: {fps:.1f} | Particles: {np.count_nonzero(self.h_types == SAND)}")
            pygame.display.flip()

        pygame.quit()
        sys.exit()

if __name__ == "__main__":
    app = SandSimulation()
    app.run()
```

```
import pygame
import random
import sys
import numpy as np
import pycuda.driver as cuda
import pycuda.autoinit
from pycuda.compiler import SourceModule

# --- Configuration ---
WINDOW_WIDTH = 1900
WINDOW_HEIGHT = 975
SIDEBAR_WIDTH = 250

# GPU Mapping
# We use a 1:1 pixel ratio for maximum detail
CELL_SIZE = 1
GRID_WIDTH = (WINDOW_WIDTH - SIDEBAR_WIDTH) // CELL_SIZE
GRID_HEIGHT = WINDOW_HEIGHT // CELL_SIZE

# UI Colors (Neutral/Dark theme)
COLOR_BG = (20, 20, 25)
COLOR_SIDEBAR = (35, 35, 40)
COLOR_BUTTON = (50, 50, 60)
COLOR_BUTTON_HOVER = (70, 70, 80)
COLOR_BUTTON_ACTIVE = (100, 120, 140)
COLOR_TEXT = (220, 220, 220)

# Sand Palette
SAND_COLORS = [
    (194, 178, 128), (230, 100, 100), (100, 180, 230),
    (150, 230, 150), (230, 230, 150), (200, 100, 200),
    (230, 230, 230)
]

# Element Types
EMPTY = 0
WALL = 1
SAND = 2
MOVED = 3

# --- CUDA Kernels ---
cuda_code = """
// 0=Empty, 1=Wall, 2=Sand, 3=Moved

// Configuration Constants
#define MAX_VELOCITY 8.0f
#define GRAVITY 0.2f

```

```
__global__ void update_physics(
    int *types,
    int *colors,
    float *velocities,
    int width,
    int height,
    int frame,
    int parity
) {
    // 1. Thread Coordinates
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int x = blockIdx.x * blockDim.x + tx;
    int y = blockIdx.y * blockDim.y + ty;

    // --- OPTIMIZATION: SHARED MEMORY EARLY EXIT ---
    // Use shared memory to check if this 16x16 block has any sand.
    // If the block is pure air, we exit immediately.

    __shared__ bool block_active;
    if (tx == 0 && ty == 0) block_active = false;
    __syncthreads();

    // Check if this specific thread is active
    if (x < width && y < height) {
        int idx = y * width + x;
        if (types[idx] == 2) block_active = true;
    }
    __syncthreads();

    // If no sand in the entire block, exit all threads.
    if (!block_active) return;
    // ------------------------------------------------

    // Boundary & Parity Checks
    if (x >= width || y >= height - 1) return;
    if (y % 2 != parity) return; // Checkerboard update

    int idx = y * width + x;
    int type = types[idx];

    // Only process SAND (2)
    if (type == 2) {
        // 2. Physics & Velocity
        float v = velocities[idx];
        v += GRAVITY;
        if (v > MAX_VELOCITY) v = MAX_VELOCITY;

        // Determine number of pixels to move this frame (Raymarching steps)
        int steps = (int)ceil(v);

        int current_idx = idx;
        int current_x = x;
        int current_y = y;
        bool stopped = false;

        // 3. Raymarching Loop
        // Attempt to move 'steps' times in one update
        for (int s = 0; s < steps; s++) {
            if (current_y >= height - 1) {
                stopped = true;
                break;
            }
```

```
112
113             int down = current_idx + width;
114             int down_left = down - 1;
115             int down_right = down + 1;
116
117             int next_idx = -1;
118
119             // Try Down
120             if (atomicCAS(&types[down], 0, 3) == 0) {
121                 next_idx = down;
122             }
123             else {
124                 // Stochastic Logic (Randomly prefer left or right)
125                 bool try_left = ((current_x + current_y + frame) % 2 == 0);
126                 if (try_left) {
127                     if (current_x > 0 && atomicCAS(&types[down_left], 0, 3) == 0)
    next_idx = down_left;
128                     else if (current_x < width - 1 && atomicCAS(&types[down_right],
    0, 3) == 0) next_idx = down_right;
129                 } else {
130                     if (current_x < width - 1 && atomicCAS(&types[down_right], 0, 3)
    == 0) next_idx = down_right;
131                     else if (current_x > 0 && atomicCAS(&types[down_left], 0, 3) ==
    0) next_idx = down_left;
132                 }
133             }
134
135             if (next_idx != -1) {
136                 // Move Successful
137
138                 // Clear the trail behind us (the spot we just left)
139                 // If we are at the start (current_idx == idx), we don't clear yet (
    we do it at the end)
140                 if (current_idx != idx) {
141                     types[current_idx] = 0;
142                     colors[current_idx] = 0;
143                     velocities[current_idx] = 0;
144                 }
145
146                 // Update pointers
147                 current_idx = next_idx;
148                 current_y++;
149                 if (next_idx == down_left) current_x--;
150                 if (next_idx == down_right) current_x++;
151
152                 // Mark new spot as Moved (3) so others don't steal it in this frame
153                 types[current_idx] = 3;
154             } else {
155                 // Blocked
156                 stopped = true;
157                 v = 0; // Reset velocity
158                 break;
159             }
160         }
161
162         // 4. Finalize
163         if (current_idx != idx) {
164             // We moved at least once. Transfer data to final spot.
165             colors[current_idx] = colors[idx];
166             velocities[current_idx] = v; // Conserve momentum
167
168             // Clear original spot
169             types[idx] = 0;
```

```
            colors[idx] = 0;
            velocities[idx] = 0;
        } else {
            // Didn't move. Keep velocity? Or reset if stopped?
            if (stopped) velocities[idx] = 0;
            else velocities[idx] = v;
        }
    }
}

__global__ void cleanup_grid(int *types, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y >= height) return;

    int idx = y * width + x;

    // Reset 'Moved' (3) back to 'Sand' (2) for the next sub-step
    if (types[idx] == 3) {
        types[idx] = 2;
    }
}

__global__ void clear_bottom_row(int *types, int *colors, float *velocities, int
    width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    if (x < width) {
        int idx = (height - 1) * width + x;
        types[idx] = 0;
        colors[idx] = 0;
        velocities[idx] = 0;
    }
}
"""

# Compile CUDA Source
mod = SourceModule(cuda_code)
update_physics_gpu = mod.get_function("update_physics")
cleanup_grid_gpu = mod.get_function("cleanup_grid")
clear_bottom_row_gpu = mod.get_function("clear_bottom_row")


class Slider:
    def __init__(self, x, y, width, min_val, max_val, initial_val):
        self.rect = pygame.Rect(x, y, width, 20)
        self.min_val = min_val
        self.max_val = max_val
        self.value = initial_val
        self.dragging = False

    def draw(self, screen, font):
        pygame.draw.line(screen, (100, 100, 100), (self.rect.left, self.rect.centery)
    ,
                         (self.rect.right, self.rect.centery), 4)
        ratio = (self.value - self.min_val) / (self.max_val - self.min_val)
        pos_x = self.rect.left + ratio * self.rect.width
        knob_rect = pygame.Rect(pos_x - 6, self.rect.centery - 10, 12, 20)
        pygame.draw.rect(screen, (180, 180, 180), knob_rect, border_radius=4)
        val_surf = font.render(f"Size: {int(self.value)}", True, COLOR_TEXT)
        screen.blit(val_surf, (self.rect.right + 10, self.rect.centery - 10))

    def handle_event(self, event):
```

```
        if event.type == pygame.MOUSEBUTTONDOWN:
            if self.rect.collidepoint(event.pos):
                self.dragging = True
                self.update_value(event.pos[0])
                return True
        elif event.type == pygame.MOUSEBUTTONUP:
            self.dragging = False
        elif event.type == pygame.MOUSEMOTION:
            if self.dragging:
                self.update_value(event.pos[0])
                return True
        return False

    def update_value(self, mouse_x):
        relative_x = mouse_x - self.rect.left
        ratio = max(0, min(1, relative_x / self.rect.width))
        self.value = self.min_val + ratio * (self.max_val - self.min_val)

    def get_value(self):
        return int(self.value)


class Button:
    def __init__(self, x, y, width, height, text, callback, color=None):
        self.rect = pygame.Rect(x, y, width, height)
        self.text = text
        self.callback = callback
        self.base_color = color if color else COLOR_BUTTON
        self.hovered = False
        self.active = False

    def draw(self, screen, font):
        color = COLOR_BUTTON_ACTIVE if self.active else (COLOR_BUTTON_HOVER if self.hovered else self.base_color)
        pygame.draw.rect(screen, color, self.rect, border_radius=5)
        # Subtle border
        pygame.draw.rect(screen, (70, 70, 80), self.rect, 1, border_radius=5)

        if isinstance(self.text, tuple):
            # Draw Color Circle
            pygame.draw.circle(screen, self.text, self.rect.center, self.rect.height // 3)
            # Draw border around circle if active
            if self.active:
                pygame.draw.circle(screen, (255, 255, 255), self.rect.center, self.rect.height // 3 + 2, 2)
        else:
            text_surf = font.render(self.text, True, COLOR_TEXT)
            text_rect = text_surf.get_rect(center=self.rect.center)
            screen.blit(text_surf, text_rect)

    def check_hover(self, mouse_pos):
        self.hovered = self.rect.collidepoint(mouse_pos)

    def handle_click(self, mouse_pos):
        if self.hovered and self.callback:
            self.callback()
            return True
        return False


class SandSimulation:
    def __init__(self):
```

```
        pygame.init()
        self.screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
        pygame.display.set_caption("Falling Sand - GPU Accelerated")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("Segoe UI", 16)
        self.title_font = pygame.font.SysFont("Segoe UI", 24, bold=True)

        # --- HOST ARRAYS (CPU) ---
        self.h_types = np.zeros((GRID_HEIGHT, GRID_WIDTH), dtype=np.int32)
        self.h_colors = np.zeros((GRID_HEIGHT, GRID_WIDTH), dtype=np.int32)
        self.h_velocities = np.zeros((GRID_HEIGHT, GRID_WIDTH), dtype=np.float32)

        # --- DEVICE ARRAYS (GPU) ---
        self.d_types = cuda.mem_alloc(self.h_types.nbytes)
        self.d_colors = cuda.mem_alloc(self.h_colors.nbytes)
        self.d_velocities = cuda.mem_alloc(self.h_velocities.nbytes)

        # Initial Copy
        cuda.memcpy_htod(self.d_types, self.h_types)
        cuda.memcpy_htod(self.d_colors, self.h_colors)
        cuda.memcpy_htod(self.d_velocities, self.h_velocities)

        # CUDA Dimensions
        block_options = [
            (4, 4, 1),
            (4, 8, 1),
            (8, 8, 1),
            (16, 8, 1),
            (16, 16, 1),
            (32, 8, 1),
            (32, 16, 1),
            (32, 32, 1),  # max 1024 threads
        ]


        self.block_dim = block_options[3]
        self.grid_dim = (
            (GRID_WIDTH + self.block_dim[0] - 1) // self.block_dim[0],
            (GRID_HEIGHT + self.block_dim[1] - 1) // self.block_dim[1]
        )

        # App State
        self.running = True
        self.simulation_active = True
        self.raining = False
        self.void_floor = False
        self.current_tool = SAND
        self.current_color = SAND_COLORS[0]
        self.brush_size = 8
        self.frame_count = 0

        self.buttons = []
        self.slider = None
        self.init_ui()

    def pack_color(self, rgb):
        return (rgb[0] << 16) | (rgb[1] << 8) | rgb[2]

    def init_ui(self):
        x_pad = 20
        y_start = 80
        btn_w = SIDEBAR_WIDTH - 40
        btn_h = 40
```

```
354
355         # Tools
356         self.buttons.append(Button(x_pad, y_start, btn_w, btn_h, "Tool: Sand", lambda
    : self.set_tool(SAND)))
357         self.buttons.append(Button(x_pad, y_start + 50, btn_w, btn_h, "Tool: Wall",
    lambda: self.set_tool(WALL)))
358         self.buttons.append(Button(x_pad, y_start + 100, btn_w, btn_h, "Tool: Eraser"
    , lambda: self.set_tool(EMPTY)))
359
360         # Slider
361         self.slider = Slider(x_pad + 5, y_start + 160, 150, 1, 100, self.brush_size)
362
363         # Colors
364         y_colors = y_start + 200
365         swatch_s = (btn_w - 10) // 4
366         for i, color in enumerate(SAND_COLORS):
367             r, c = divmod(i, 4)
368             btn = Button(x_pad + c * (swatch_s + 5), y_colors + r * (swatch_s + 5),
    swatch_s, swatch_s, color, lambda c=color: self.set_color(c))
369             self.buttons.append(btn)
370
371         # Controls
372         y_ctrl = y_colors + 150
373         self.btn_rain = Button(x_pad, y_ctrl, btn_w, btn_h, "Auto-Flow: Off", self.
    toggle_rain)
374         self.buttons.append(self.btn_rain)
375
376         self.btn_void = Button(x_pad, y_ctrl + 50, btn_w, btn_h, "Void Floor: Off",
    self.toggle_void)
377         self.buttons.append(self.btn_void)
378
379         self.btn_pause = Button(x_pad, y_ctrl + 100, btn_w, btn_h, "Pause (Space)",
    self.toggle_pause)
380         self.buttons.append(self.btn_pause)
381
382         self.buttons.append(Button(x_pad, y_ctrl + 150, btn_w, btn_h, "Clear Grid (R)
    ", self.reset_grid))
383
384         self.update_button_states()
385
386     def set_tool(self, tool):
387         self.current_tool = tool
388         self.update_button_states()
389
390     def set_color(self, color):
391         self.current_color = color
392         self.current_tool = SAND
393         self.update_button_states()
394
395     def toggle_pause(self):
396         self.simulation_active = not self.simulation_active
397         self.btn_pause.text = "Play (Space)" if not self.simulation_active else "
    Pause (Space)"
398         self.update_button_states()
399
400     def toggle_rain(self):
401         self.raining = not self.raining
402         self.btn_rain.text = "Auto-Flow: On" if self.raining else "Auto-Flow: Off"
403         self.btn_rain.active = self.raining
404
405     def toggle_void(self):
406         self.void_floor = not self.void_floor
407         self.btn_void.text = "Void Floor: On" if self.void_floor else "Void Floor:
```

```
    Off"
408         self.btn_void.active = self.void_floor
409 
410     def update_button_states(self):
411         for btn in self.buttons:
412             btn.active = False
413             if "Tool: Sand" in str(btn.text) and self.current_tool == SAND: btn.
    active = True
414             if "Tool: Wall" in str(btn.text) and self.current_tool == WALL: btn.
    active = True
415             if "Tool: Eraser" in str(btn.text) and self.current_tool == EMPTY: btn.
    active = True
416             if isinstance(btn.text, tuple) and btn.text == self.current_color: btn.
    active = True
417             if btn == self.btn_rain and self.raining: btn.active = True
418             if btn == self.btn_void and self.void_floor: btn.active = True
419 
420     def reset_grid(self):
421         self.h_types.fill(EMPTY)
422         self.h_colors.fill(0)
423         self.h_velocities.fill(0)
424         cuda.memcpy_htod(self.d_types, self.h_types)
425         cuda.memcpy_htod(self.d_colors, self.h_colors)
426         cuda.memcpy_htod(self.d_velocities, self.h_velocities)
427 
428     def draw_brush(self, mx, my):
429         gx = (mx - SIDEBAR_WIDTH) // CELL_SIZE
430         gy = my // CELL_SIZE
431         r = self.brush_size
432 
433         if not (0 <= gx < GRID_WIDTH and 0 <= gy < GRID_HEIGHT): return
434 
435         x_min = max(0, gx - r)
436         x_max = min(GRID_WIDTH, gx + r + 1)
437         y_min = max(0, gy - r)
438         y_max = min(GRID_HEIGHT, gy + r + 1)
439 
440         y_idx, x_idx = np.ogrid[y_min:y_max, x_min:x_max]
441         mask = (x_idx - gx)**2 + (y_idx - gy)**2 <= r**2
442 
443         if self.current_tool == SAND:
444             base = self.current_color
445             noise = np.random.randint(-15, 15, (y_max-y_min, x_max-x_min, 3))
446             rgb_slice = np.clip(np.array(base) + noise, 0, 255).astype(np.int32)
447             packed_colors = (rgb_slice[...,0] << 16) | (rgb_slice[...,1] << 8) |
    rgb_slice[...,2]
448 
449             density = np.random.random(mask.shape) < 0.7
450             final_mask = mask & density
451 
452             self.h_types[y_min:y_max, x_min:x_max][final_mask] = SAND
453             self.h_colors[y_min:y_max, x_min:x_max][final_mask] = packed_colors[
    final_mask]
454             # Important: Reset velocity for new sand
455             self.h_velocities[y_min:y_max, x_min:x_max][final_mask] = 0
456 
457         elif self.current_tool == WALL:
458             self.h_types[y_min:y_max, x_min:x_max][mask] = WALL
459             self.h_colors[y_min:y_max, x_min:x_max][mask] = self.pack_color((120,
    120, 130))
460             self.h_velocities[y_min:y_max, x_min:x_max][mask] = 0
461 
462         else: # Eraser
```

```
            self.h_types[y_min:y_max, x_min:x_max][mask] = EMPTY
            self.h_colors[y_min:y_max, x_min:x_max][mask] = 0
            self.h_velocities[y_min:y_max, x_min:x_max][mask] = 0

        # Upload
        cuda.memcpy_htod(self.d_types, self.h_types)
        cuda.memcpy_htod(self.d_colors, self.h_colors)
        cuda.memcpy_htod(self.d_velocities, self.h_velocities)

    def update(self):
        # 1. Rain Logic
        if self.raining:
            drops = 200
            xs = np.random.randint(0, GRID_WIDTH, drops)
            valid = self.h_types[0, xs] == EMPTY
            valid_xs = xs[valid]
            if len(valid_xs) > 0:
                self.h_types[0, valid_xs] = SAND
                base = np.array(self.current_color)
                noise = np.random.randint(-20, 20, (len(valid_xs), 3))
                col = np.clip(base + noise, 0, 255).astype(np.int32)
                packed = (col[:,0] << 16) | (col[:,1] << 8) | col[:,2]
                self.h_colors[0, valid_xs] = packed
                self.h_velocities[0, valid_xs] = 0

                cuda.memcpy_htod(self.d_types, self.h_types)
                cuda.memcpy_htod(self.d_colors, self.h_colors)
                cuda.memcpy_htod(self.d_velocities, self.h_velocities)

        # 2. Physics Execution
        if self.simulation_active:
            # Sub-steps: With gravity/velocity, we can reduce sub-steps to 2-4
            # and still get a solid feel because particles move >1 pixel per step.
            SUB_STEPS = 2

            for _ in range(SUB_STEPS):
                # Checkerboard Update: Even rows then Odd rows
                for parity in range(2):
                    update_physics_gpu(
                        self.d_types, self.d_colors, self.d_velocities,
                        np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT), np.int32(self.
    frame_count),
                        np.int32(parity),
                        block=self.block_dim, grid=self.grid_dim
                    )

                # Cleanup Moved Status (3 -> 2)
                cleanup_grid_gpu(
                    self.d_types,
                    np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT),
                    block=self.block_dim, grid=self.grid_dim
                )

            # Void Floor (Clear bottom)
            if self.void_floor:
                block_x = 256
                grid_x = (GRID_WIDTH + block_x - 1) // block_x
                clear_bottom_row_gpu(
                    self.d_types, self.d_colors, self.d_velocities,
                    np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT),
                    block=(block_x, 1, 1), grid=(grid_x, 1)
                )

```

```
        # 3. Download Results
        cuda.memcpy_dtoh(self.h_types, self.d_types)
        cuda.memcpy_dtoh(self.h_colors, self.d_colors)

        self.frame_count += 1

    def run(self):
        while self.running:
            # Important: Call tick() without args to track FPS without capping
            self.clock.tick()

            for event in pygame.event.get():
                if event.type == pygame.QUIT: self.running = False

                if self.slider.handle_event(event):
                    self.brush_size = self.slider.get_value()

                elif event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_SPACE: self.toggle_pause()
                    elif event.key == pygame.K_r: self.reset_grid()

                elif event.type == pygame.MOUSEBUTTONDOWN:
                    if event.button == 1 and event.pos[0] < SIDEBAR_WIDTH:
                        for btn in self.buttons:
                            btn.handle_click(event.pos)

            # Mouse Interactions
            m_buttons = pygame.mouse.get_pressed()
            m_pos = pygame.mouse.get_pos()
            if m_pos[0] > SIDEBAR_WIDTH:
                if m_buttons[0]: self.draw_brush(m_pos[0], m_pos[1])
                elif m_buttons[2]: # Right click erase
                    prev, self.current_tool = self.current_tool, EMPTY
                    self.draw_brush(m_pos[0], m_pos[1])
                    self.current_tool = prev

            # Update Sim
            self.update()

            # Render
            self.screen.fill(COLOR_BG)

            # Draw Sidebar
            pygame.draw.rect(self.screen, COLOR_SIDEBAR, (0, 0, SIDEBAR_WIDTH,
    WINDOW_HEIGHT))
            title = self.title_font.render("GPU Sand Sim", True, (255, 255, 255))
            self.screen.blit(title, (20, 20))

            for btn in self.buttons:
                btn.check_hover(m_pos)
                btn.draw(self.screen, self.font)
            self.slider.draw(self.screen, self.font)

            # Draw Grid
            # Transpose for Pygame Surfarray: (H, W) -> (W, H)
            surface_array = self.h_colors.T
            if not hasattr(self, 'grid_surface'):
                self.grid_surface = pygame.Surface((GRID_WIDTH, GRID_HEIGHT))

            pygame.surfarray.blit_array(self.grid_surface, surface_array)
            self.screen.blit(self.grid_surface, (SIDEBAR_WIDTH, 0))

            # Update FPS Caption periodically (every 30 frames to save CPU)
```

```
            if self.frame_count % 30 == 0:
                fps = self.clock.get_fps()
                # Use a rough estimate for particle count if exact count is too slow
                # or just count every now and then
                count = np.count_nonzero(self.h_types == SAND)
                pygame.display.set_caption(f"GPU Sand | FPS: {fps:.1f} | Particles: {count}")

            pygame.display.flip()

        pygame.quit()
        sys.exit()

if __name__ == "__main__":
    app = SandSimulation()
    app.run()
```

```
import pygame
import random
import sys
import numpy as np
import pycuda.driver as cuda
import pycuda.autoinit
from pycuda.compiler import SourceModule

# --- Configuration ---
WINDOW_WIDTH = 1900
WINDOW_HEIGHT = 975
SIDEBAR_WIDTH = 300

# Physics Grid Configuration
CELL_SIZE = 1
GRID_WIDTH = (WINDOW_WIDTH - SIDEBAR_WIDTH) // CELL_SIZE
GRID_HEIGHT = WINDOW_HEIGHT // CELL_SIZE

# Colors
COLOR_BG = (20, 20, 25)
COLOR_SIDEBAR = (40, 40, 45)
COLOR_BUTTON = (60, 60, 70)
COLOR_BUTTON_HOVER = (80, 80, 90)
COLOR_BUTTON_ACTIVE = (100, 100, 120)
COLOR_TEXT = (200, 200, 200)

# Sand Palette
SAND_COLORS = [
    (194, 178, 128), (230, 100, 100), (100, 180, 230),
    (150, 230, 150), (230, 230, 150), (200, 100, 200),
    (230, 230, 230)
]

# Element Types (Integers for GPU)
EMPTY = 0
WALL = 1
SAND = 2
BEAM = 4 # Special tool ID

SUB_STEPS=1

# --- CUDA Kernels ---
cuda_code = """
// --- PHYSICS KERNEL ---
__global__ void update_physics(int *types, int *colors, int width, int height, int frame) {
    int tx = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    int ty = blockIdx.y * blockDim.y + threadIdx.y;

    // FLIP Y: Process bottom-up
    int x = tx;
    int y = (height - 1) - ty;

    if (x >= width || y < 0) return;

    int idx = y * width + x;
    int type = types[idx];

    if (type == 2) {
        if (y >= height - 1) return;

        int down = idx + width;
        int down_left = down - 1;
        int down_right = down + 1;

        if (atomicCAS(&types[down], 0, 3) == 0) {
            colors[down] = colors[idx];
            types[idx] = 0;
            colors[idx] = 0;
            return;
        }

        bool try_left_first = ((x + y + frame) % 2 == 0);

        if (try_left_first) {
            if (x > 0 && atomicCAS(&types[down_left], 0, 3) == 0) {
                colors[down_left] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            } else if (x < width - 1 && atomicCAS(&types[down_right], 0, 3) == 0) {
                colors[down_right] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
        } else {
            if (x < width - 1 && atomicCAS(&types[down_right], 0, 3) == 0) {
                colors[down_right] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            } else if (x > 0 && atomicCAS(&types[down_left], 0, 3) == 0) {
                colors[down_left] = colors[idx];
                types[idx] = 0;
                colors[idx] = 0;
            }
        }
    }
}

// --- CLEANUP KERNEL ---
__global__ void cleanup_grid(int *types, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y >= height) return;

    int idx = y * width + x;
    if (types[idx] == 3) {
        types[idx] = 2;
    }
}
```

```
110
111 // --- VOID FLOOR KERNEL ---
112 __global__ void clear_bottom_row(int *types, int *colors, int width, int height) {
113     int x = blockIdx.x * blockDim.x + threadIdx.x;
114     if (x < width) {
115         int idx = (height - 1) * width + x;
116         types[idx] = 0;
117         colors[idx] = 0;
118     }
119 }
120
121 // --- NEW: COALESCED BEAM GENERATOR ---
122 // This kernel fills a horizontal line.
123 // Since grid is row-major, x and x+1 are adjacent in memory.
124 // Threads (tid) and (tid+1) write to adjacent addresses.
125 // This is the Definition of Coalesced Memory Access.
126 __global__ void generate_beam(int *types, int *colors, int end_x, int y, int width,
    int color) {
127     int x = blockIdx.x * blockDim.x + threadIdx.x;
128
129     // Safety check
130     if (x >= end_x || x >= width) return;
131
132     int idx = y * width + x;
133
134     // Only overwrite empty space to prevent flickering walls
135     if (types[idx] == 0) {
136         types[idx] = 2; // Create Sand
137         colors[idx] = color;
138     }
139 }
140 """
141
142 mod = SourceModule(cuda_code)
143 update_physics_gpu = mod.get_function("update_physics")
144 cleanup_grid_gpu = mod.get_function("cleanup_grid")
145 clear_bottom_row_gpu = mod.get_function("clear_bottom_row")
146 generate_beam_gpu = mod.get_function("generate_beam") # <--- New Function
147
148 class Slider:
149     def __init__(self, x, y, width, min_val, max_val, initial_val):
150         self.rect = pygame.Rect(x, y, width, 20)
151         self.min_val = min_val
152         self.max_val = max_val
153         self.value = initial_val
154         self.dragging = False
155
156     def draw(self, screen, font):
157         pygame.draw.line(screen, (100, 100, 100), (self.rect.left, self.rect.centery)
    ,
158                          (self.rect.right, self.rect.centery), 4)
159         ratio = (self.value - self.min_val) / (self.max_val - self.min_val)
160         pos_x = self.rect.left + ratio * self.rect.width
161         knob_rect = pygame.Rect(pos_x - 6, self.rect.centery - 10, 12, 20)
162         pygame.draw.rect(screen, (180, 180, 180), knob_rect, border_radius=4)
163         val_surf = font.render(f"Size: {int(self.value)}", True, COLOR_TEXT)
164         screen.blit(val_surf, (self.rect.right + 10, self.rect.centery - 10))
165
166     def handle_event(self, event):
167         if event.type == pygame.MOUSEBUTTONDOWN:
168             if self.rect.collidepoint(event.pos):
169                 self.dragging = True
170                 self.update_value(event.pos[0])
```

```
                return True
        elif event.type == pygame.MOUSEBUTTONUP:
            self.dragging = False
        elif event.type == pygame.MOUSEMOTION:
            if self.dragging:
                self.update_value(event.pos[0])
                return True
        return False

    def update_value(self, mouse_x):
        relative_x = mouse_x - self.rect.left
        ratio = max(0, min(1, relative_x / self.rect.width))
        self.value = self.min_val + ratio * (self.max_val - self.min_val)

    def get_value(self):
        return int(self.value)


class Button:
    def __init__(self, x, y, width, height, text, callback, color=None):
        self.rect = pygame.Rect(x, y, width, height)
        self.text = text
        self.callback = callback
        self.base_color = color if color else COLOR_BUTTON
        self.hovered = False
        self.active = False

    def draw(self, screen, font):
        color = COLOR_BUTTON_ACTIVE if self.active else (COLOR_BUTTON_HOVER if self.
    hovered else self.base_color)
        pygame.draw.rect(screen, color, self.rect, border_radius=5)
        pygame.draw.rect(screen, (30, 30, 30), self.rect, 2, border_radius=5)

        if isinstance(self.text, tuple):
            pygame.draw.circle(screen, self.text, self.rect.center, self.rect.height
    // 3)
        else:
            text_surf = font.render(self.text, True, COLOR_TEXT)
            text_rect = text_surf.get_rect(center=self.rect.center)
            screen.blit(text_surf, text_rect)

    def check_hover(self, mouse_pos):
        self.hovered = self.rect.collidepoint(mouse_pos)

    def handle_click(self, mouse_pos):
        if self.hovered and self.callback:
            self.callback()
            return True
        return False


class SandSimulation:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
        pygame.display.set_caption("Falling Sand - GPU Accelerated")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("Arial", 16)
        self.title_font = pygame.font.SysFont("Arial", 22, bold=True)

        self.h_types = np.zeros((GRID_HEIGHT, GRID_WIDTH), dtype=np.int32)
        self.h_colors = np.zeros((GRID_HEIGHT, GRID_WIDTH), dtype=np.int32)

```

```
        self.d_types = cuda.mem_alloc(self.h_types.nbytes)
        self.d_colors = cuda.mem_alloc(self.h_colors.nbytes)

        cuda.memcpy_htod(self.d_types, self.h_types)
        cuda.memcpy_htod(self.d_colors, self.h_colors)

        self.block_dim = (16, 8, 1)
        self.grid_dim = (
            (GRID_WIDTH + self.block_dim[0] - 1) // self.block_dim[0],
            (GRID_HEIGHT + self.block_dim[1] - 1) // self.block_dim[1]
        )

        self.running = True
        self.simulation_active = True
        self.raining = False
        self.void_floor = False
        self.current_tool = SAND
        self.current_color = SAND_COLORS[0]
        self.brush_size = 5
        self.frame_count = 0

        self.buttons = []
        self.slider = None
        self.init_ui()

    def pack_color(self, rgb):
        return (rgb[0] << 16) | (rgb[1] << 8) | rgb[2]

    def init_ui(self):
        x_pad = 10
        y_start = 60
        btn_w = SIDEBAR_WIDTH - 20
        btn_h = 35

        self.buttons.append(Button(x_pad, y_start, btn_w, btn_h, "Tool: Sand", lambda
    : self.set_tool(SAND)))
        self.buttons.append(Button(x_pad, y_start + 45, btn_w, btn_h, "Tool: Wall",
    lambda: self.set_tool(WALL)))
        self.buttons.append(Button(x_pad, y_start + 90, btn_w, btn_h, "Tool: Eraser",
     lambda: self.set_tool(EMPTY)))
        # NEW BUTTON
        self.buttons.append(Button(x_pad, y_start + 135, btn_w, btn_h, "Tool: Beam (
    Coalesced)", lambda: self.set_tool(BEAM)))

        self.slider = Slider(x_pad + 5, y_start + 190, 150, 1, 100, self.brush_size)

        y_colors = y_start + 235
        swatch_s = (btn_w - 10) // 3
        for i, color in enumerate(SAND_COLORS):
            r, c = divmod(i, 3)
            btn = Button(x_pad + c * (swatch_s + 5), y_colors + r * (swatch_s + 5),
    swatch_s, swatch_s, color, lambda c=color: self.set_color(c))
            self.buttons.append(btn)

        y_ctrl = y_colors + 150
        self.btn_rain = Button(x_pad, y_ctrl, btn_w, btn_h, "Auto-Flow: Off", self.
    toggle_rain)
        self.buttons.append(self.btn_rain)

        self.btn_void = Button(x_pad, y_ctrl + 45, btn_w, btn_h, "Void Floor: Off",
    self.toggle_void)
        self.buttons.append(self.btn_void)

```

```
        self.btn_pause = Button(x_pad, y_ctrl + 90, btn_w, btn_h, "Pause (Space)", self.toggle_pause)
        self.buttons.append(self.btn_pause)
        self.buttons.append(Button(x_pad, y_ctrl + 135, btn_w, btn_h, "Clear (R)", self.reset_grid))

        self.update_button_states()

    def set_tool(self, tool):
        self.current_tool = tool
        self.update_button_states()

    def set_color(self, color):
        self.current_color = color
        self.current_tool = SAND
        self.update_button_states()

    def toggle_pause(self):
        self.simulation_active = not self.simulation_active
        self.btn_pause.text = "Play (Space)" if not self.simulation_active else "Pause (Space)"
        self.update_button_states()

    def toggle_rain(self):
        self.raining = not self.raining
        self.btn_rain.text = "Auto-Flow: On" if self.raining else "Auto-Flow: Off"
        self.btn_rain.active = self.raining

    def toggle_void(self):
        self.void_floor = not self.void_floor
        self.btn_void.text = "Void Floor: On" if self.void_floor else "Void Floor: Off"
        self.btn_void.active = self.void_floor

    def update_button_states(self):
        for btn in self.buttons:
            btn.active = False
            if "Tool: Sand" in str(btn.text) and self.current_tool == SAND: btn.active = True
            if "Tool: Wall" in str(btn.text) and self.current_tool == WALL: btn.active = True
            if "Tool: Eraser" in str(btn.text) and self.current_tool == EMPTY: btn.active = True
            if "Tool: Beam" in str(btn.text) and self.current_tool == BEAM: btn.active = True
            if isinstance(btn.text, tuple) and btn.text == self.current_color: btn.active = True
            if btn == self.btn_rain and self.raining: btn.active = True
            if btn == self.btn_void and self.void_floor: btn.active = True

    def reset_grid(self):
        self.h_types.fill(EMPTY)
        self.h_colors.fill(0)
        cuda.memcpy_htod(self.d_types, self.h_types)
        cuda.memcpy_htod(self.d_colors, self.h_colors)

    def draw_brush(self, mx, my):
        # We draw on CPU array then copy to GPU for standard brushes
        gx = (mx - SIDEBAR_WIDTH) // CELL_SIZE
        gy = my // CELL_SIZE
        r = self.brush_size

        if not (0 <= gx < GRID_WIDTH and 0 <= gy < GRID_HEIGHT): return
```

```
342
343         x_min = max(0, gx - r)
344         x_max = min(GRID_WIDTH, gx + r + 1)
345         y_min = max(0, gy - r)
346         y_max = min(GRID_HEIGHT, gy + r + 1)
347
348         y_idx, x_idx = np.ogrid[y_min:y_max, x_min:x_max]
349         mask = (x_idx - gx)**2 + (y_idx - gy)**2 <= r**2
350
351         if self.current_tool == SAND:
352             base = self.current_color
353             noise = np.random.randint(-20, 20, (y_max-y_min, x_max-x_min, 3))
354             rgb_slice = np.clip(np.array(base) + noise, 0, 255).astype(np.int32)
355             packed_colors = (rgb_slice[...,0] << 16) | (rgb_slice[...,1] << 8) |
    rgb_slice[...,2]
356             density = np.random.random(mask.shape) < 0.8
357             final_mask = mask & density
358             self.h_types[y_min:y_max, x_min:x_max][final_mask] = SAND
359             self.h_colors[y_min:y_max, x_min:x_max][final_mask] = packed_colors[
    final_mask]
360
361         elif self.current_tool == WALL:
362             self.h_types[y_min:y_max, x_min:x_max][mask] = WALL
363             self.h_colors[y_min:y_max, x_min:x_max][mask] = self.pack_color((120,
    120, 120))
364
365         else: # Eraser
366             self.h_types[y_min:y_max, x_min:x_max][mask] = EMPTY
367             self.h_colors[y_min:y_max, x_min:x_max][mask] = 0
368
369         cuda.memcpy_htod(self.d_types, self.h_types)
370         cuda.memcpy_htod(self.d_colors, self.h_colors)
371
372     def draw_beam_gpu(self, mx, my):
373         # This function launches the COALESCED KERNEL
374         gx = (mx - SIDEBAR_WIDTH) // CELL_SIZE
375         gy = my // CELL_SIZE
376
377         if not (0 <= gx < GRID_WIDTH and 0 <= gy < GRID_HEIGHT): return
378
379         # Calculate Beam Color
380         base = self.current_color
381         # Add a little noise for the beam
382         r = min(255, base[0] + random.randint(-10, 10))
383         g = min(255, base[1] + random.randint(-10, 10))
384         b = min(255, base[2] + random.randint(-10, 10))
385         packed_color = (r << 16) | (g << 8) | b
386
387         # Grid Calculation for 1D Line
388         # We want to fill from x=0 to x=gx
389         length = gx
390         block_size = 128
391         grid_size = (length + block_size - 1) // block_size
392
393         # Launch Kernel
394         generate_beam_gpu(
395             self.d_types, self.d_colors,
396             np.int32(length), np.int32(gy), np.int32(GRID_WIDTH), np.int32(
    packed_color),
397             block=(block_size, 1, 1), grid=(grid_size, 1)
398         )
399
400     def update(self):
```

```
        if self.raining:
            drops = 200
            xs = np.random.randint(0, GRID_WIDTH, drops)
            valid = self.h_types[0, xs] == EMPTY
            valid_xs = xs[valid]
            if len(valid_xs) > 0:
                self.h_types[0, valid_xs] = SAND
                base = np.array(self.current_color)
                noise = np.random.randint(-20, 20, (len(valid_xs), 3))
                col = np.clip(base + noise, 0, 255).astype(np.int32)
                packed = (col[:,0] << 16) | (col[:,1] << 8) | col[:,2]
                self.h_colors[0, valid_xs] = packed
                cuda.memcpy_htod(self.d_types, self.h_types)
                cuda.memcpy_htod(self.d_colors, self.h_colors)

        if self.simulation_active:
            SUB_STEPS = 8
            for _ in range(SUB_STEPS):
                # Physics Kernel
                update_physics_gpu(
                    self.d_types, self.d_colors,
                    np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT), np.int32(self.
    frame_count),
                    block=self.block_dim, grid=self.grid_dim
                )

                # Cleanup
                cleanup_grid_gpu(
                    self.d_types,
                    np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT),
                    block=self.block_dim, grid=self.grid_dim
                )

            if self.void_floor:
                block_x = 64
                grid_x = (GRID_WIDTH + block_x - 1) // block_x
                clear_bottom_row_gpu(
                    self.d_types, self.d_colors,
                    np.int32(GRID_WIDTH), np.int32(GRID_HEIGHT),
                    block=(block_x, 1, 1), grid=(grid_x, 1)
                )

        cuda.memcpy_dtoh(self.h_types, self.d_types)
        cuda.memcpy_dtoh(self.h_colors, self.d_colors)

        self.frame_count += 1

    def run(self):
        while self.running:
            self.clock.tick()

            for event in pygame.event.get():
                if event.type == pygame.QUIT: self.running = False
                if self.slider.handle_event(event): self.brush_size = self.slider.
    get_value()
                elif event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_SPACE: self.toggle_pause()
                    elif event.key == pygame.K_r: self.reset_grid()
                elif event.type == pygame.MOUSEBUTTONDOWN:
                    if event.button == 1 and event.pos[0] < SIDEBAR_WIDTH:
                        for btn in self.buttons: btn.handle_click(event.pos)

            m_buttons = pygame.mouse.get_pressed()
```

```
            m_pos = pygame.mouse.get_pos()
            if m_pos[0] > SIDEBAR_WIDTH:
                if m_buttons[0]:
                    if self.current_tool == BEAM:
                        self.draw_beam_gpu(m_pos[0], m_pos[1])
                    else:
                        self.draw_brush(m_pos[0], m_pos[1])
                elif m_buttons[2]:
                    prev, self.current_tool = self.current_tool, EMPTY
                    self.draw_brush(m_pos[0], m_pos[1])
                    self.current_tool = prev

            self.update()

            self.screen.fill(COLOR_BG)
            pygame.draw.rect(self.screen, COLOR_SIDEBAR, (0, 0, SIDEBAR_WIDTH,
    WINDOW_HEIGHT))
            title = self.title_font.render("GPU Sand Sim", True, (255, 255, 255))
            self.screen.blit(title, (20, 20))

            for btn in self.buttons:
                btn.check_hover(m_pos)
                btn.draw(self.screen, self.font)
            self.slider.draw(self.screen, self.font)

            surface_array = self.h_colors.T
            if not hasattr(self, 'grid_surface'): self.grid_surface = pygame.Surface
    ((GRID_WIDTH, GRID_HEIGHT))
            pygame.surfarray.blit_array(self.grid_surface, surface_array)
            self.screen.blit(self.grid_surface, (SIDEBAR_WIDTH, 0))

            fps = self.clock.get_fps()
            pygame.display.set_caption(f"GPU Sand | FPS: {fps:.1f} | Particles: {np.
    count_nonzero(self.h_types == SAND)}")
            pygame.display.flip()

        pygame.quit()
        sys.exit()

if __name__ == "__main__":
    app = SandSimulation()
    app.run()
```